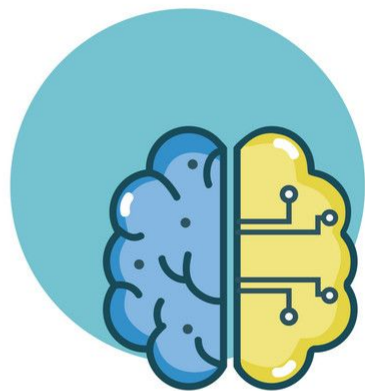


# INTRODUCTION TO MACHINE LEARNING

GENERALIZATION ERROR, MODELS, HYPOTHESIS SPACE...



Elisa Ricci



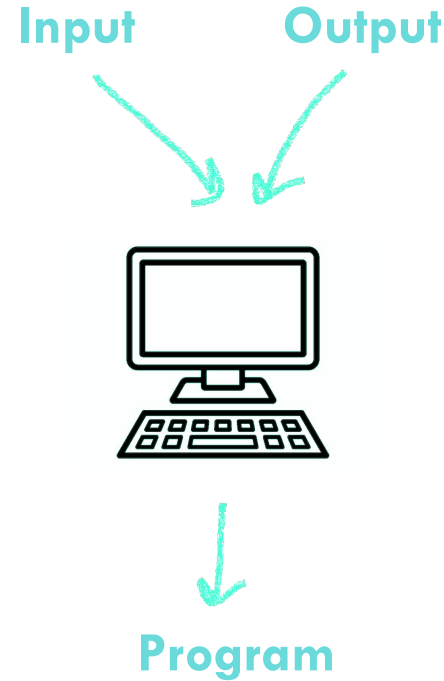
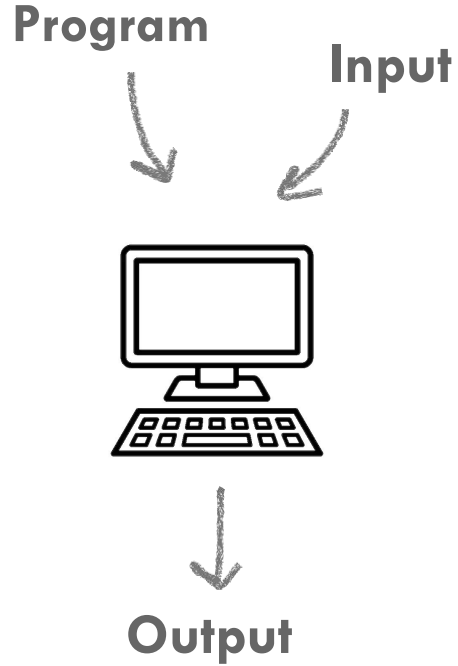
# MACHINE LEARNING IS...



**Machine learning** is the science of getting computers to act without being explicitly programmed.

A. Samuel (1959)

# PROGRAMMING VS MACHINE LEARNING



# BIRDS



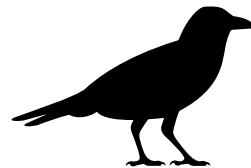
IN CS, IT CAN BE HARD TO EXPLAIN  
THE DIFFERENCE BETWEEN THE EASY  
AND THE VIRTUALLY IMPOSSIBLE.

# CLASSIFICATION

**Input**



**Output**



# SOLVING WITH CONVENTIONAL PROGRAMMING



**Almost Impossible**

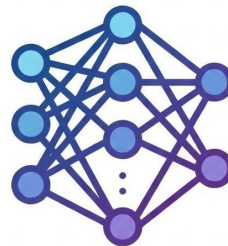
# LEARNING AT RESCUE

1



Collect  
(annotated)  
data

2



Define a family of models  
for the classification task

4

Find the model that  
minimizes the error, aka  
TRAIN or LEARN  
a model

3

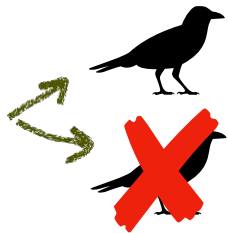


Define an error function to  
measure how well a model fits the data

# ML INGREDIENTS



TASK



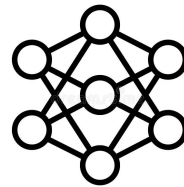
LEARNING  
ALGORITHM



DATA



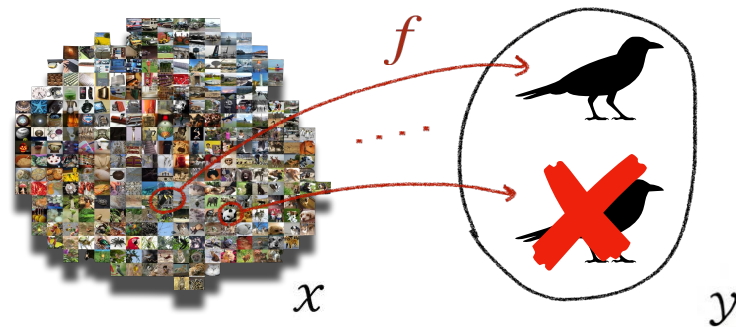
MODEL  
HYPOTHESES



OBJECTIVE



# TASK



- A **task** represents the type of prediction being made to solve a problem on some data
- We can identify a task with the **set of functions** that can potentially solve it.
- In general it consists of functions assigning each input  $x \in \mathcal{X}$  an output  $y \in \mathcal{Y}$

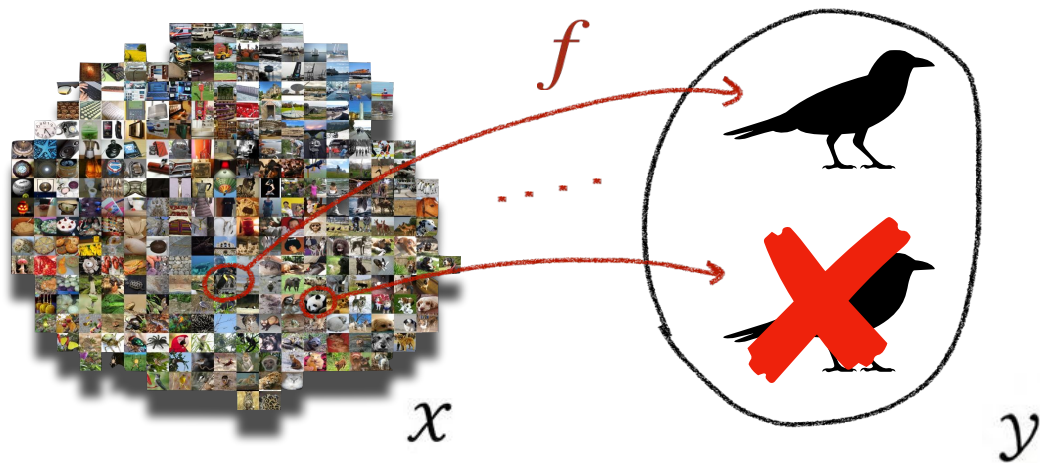
$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

- The nature of the input and the output space depends on the type of task

# CLASSIFICATION

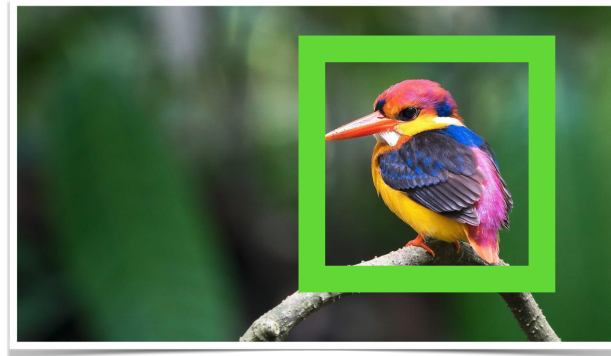
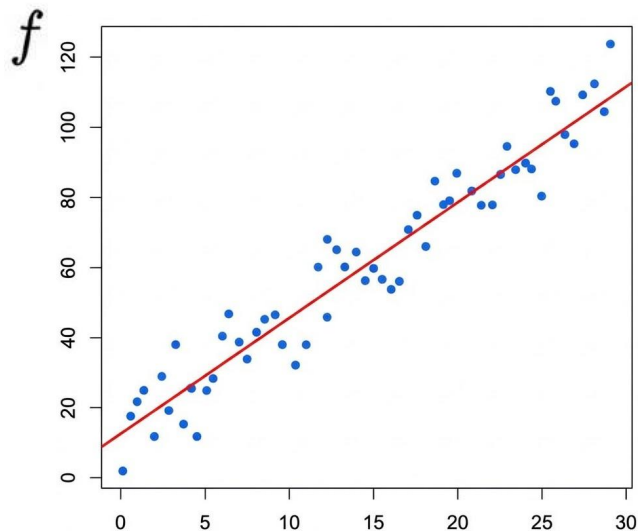
Find a function  $f$  assigning each input  $x \in \mathcal{X}$  a **discrete** label

$$f(x) \in \mathcal{Y} = \{1, \dots, k\}$$



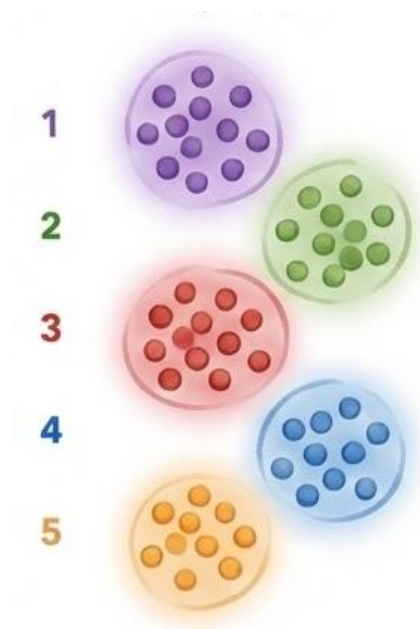
# REGRESSION

Find a function  $f$  assigning each input a **continuous** label



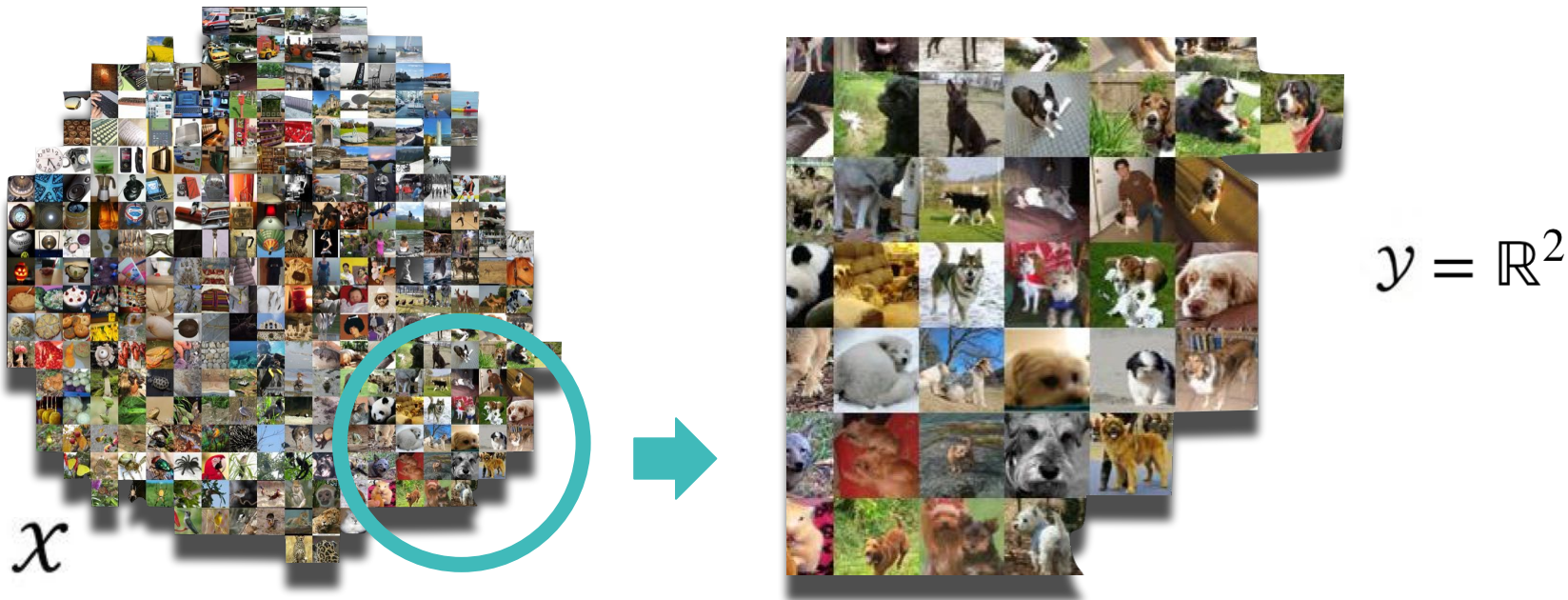
# CLUSTERING

Find a function  $f$  that assigns each input  $x \in \mathcal{X}$  a cluster index. All points mapped to the same index form a cluster.



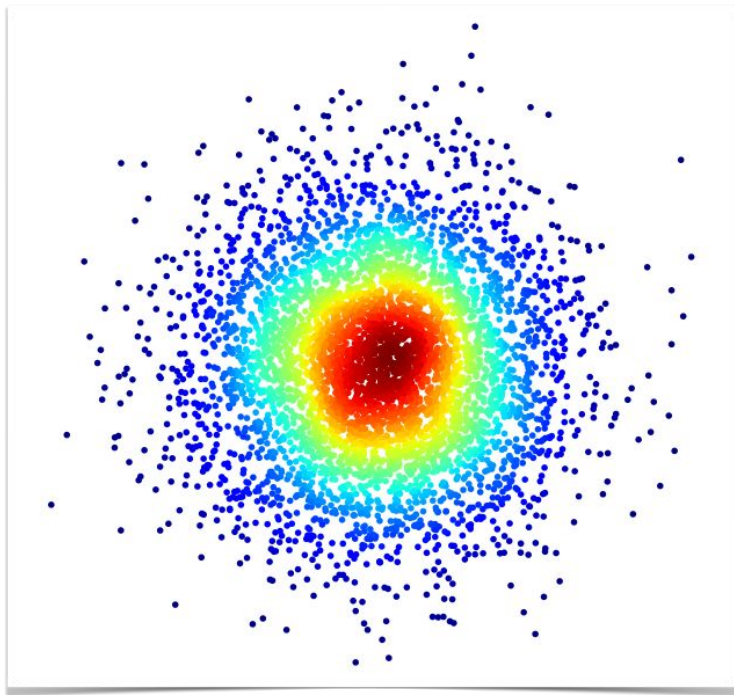
# DIMENSIONALITY REDUCTION

Find a function  $f$  mapping each (high dimensional) input  $x \in \mathcal{X}$  to a lower dimensional embedding  $y$ , where  $\dim(\mathcal{X}) \ll \dim(\mathcal{Y})$



# DENSITY ESTIMATION

Find a probability distribution  $f \in \Delta$  that fits the data

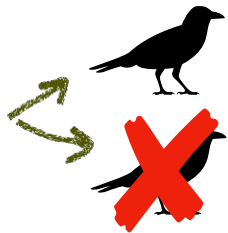


....and many other tasks

# ML INGREDIENTS



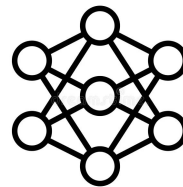
TASK



DATA



MODEL  
HYPOTHESES



LEARNING  
ALGORITHM



OBJECTIVE



# DATA



# DATA

- Information about the problem to solve
- In the form of a distribution

$$p_{\text{data}}$$

- Classification and Regression:

$$p_{\text{data}} \in \Delta(\mathcal{X} \times \mathcal{Y})$$

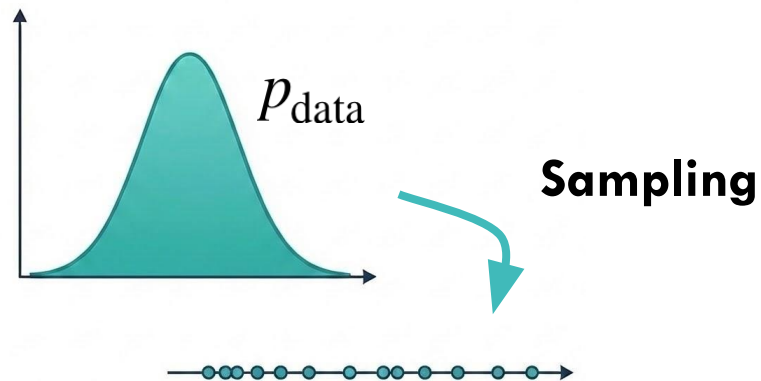
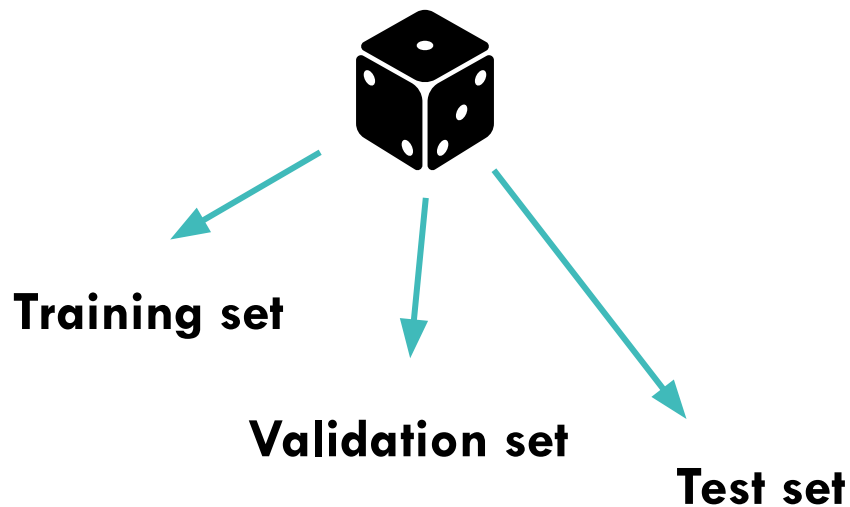
- Density estimation, Clustering and Dimensionality Reduction:

$$p_{\text{data}} \in \Delta(\mathcal{X})$$

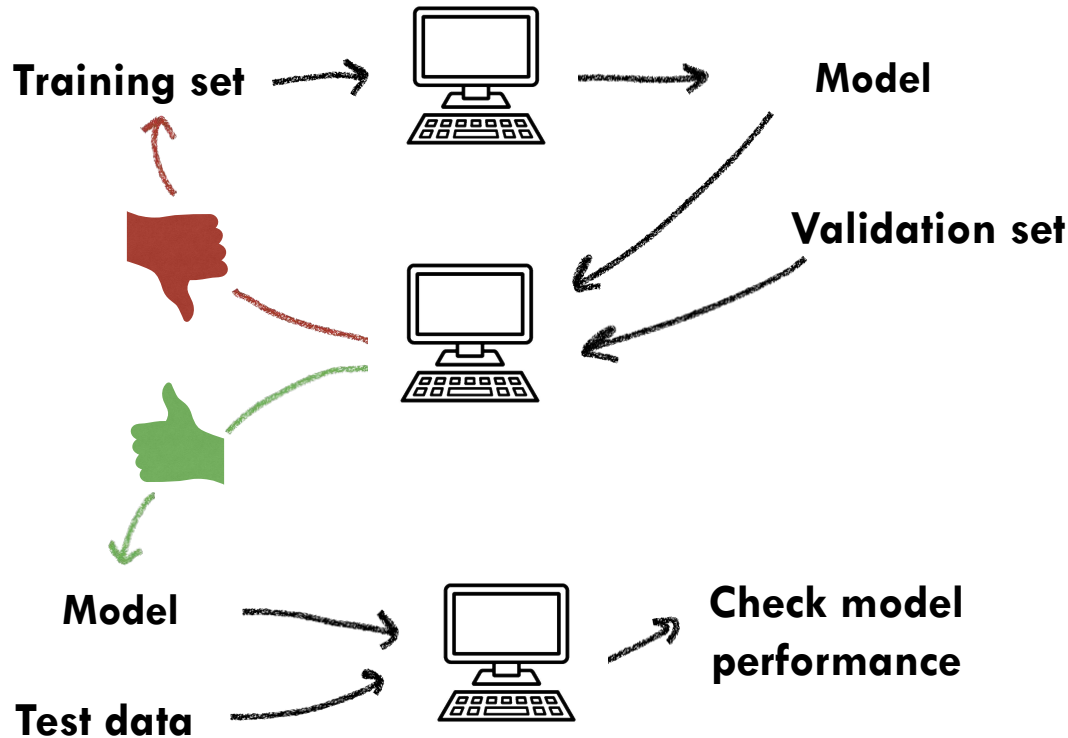


# DATA

- The data distribution  $p_{\text{data}}$  is typically unknown
- But we can sample from it



# TRAINING, VALIDATION, TEST SETS



# TRAINING & TEST SET

**Training data**



**Test set**



# TRAINING & TEST SET

**Training data**



**Test set**



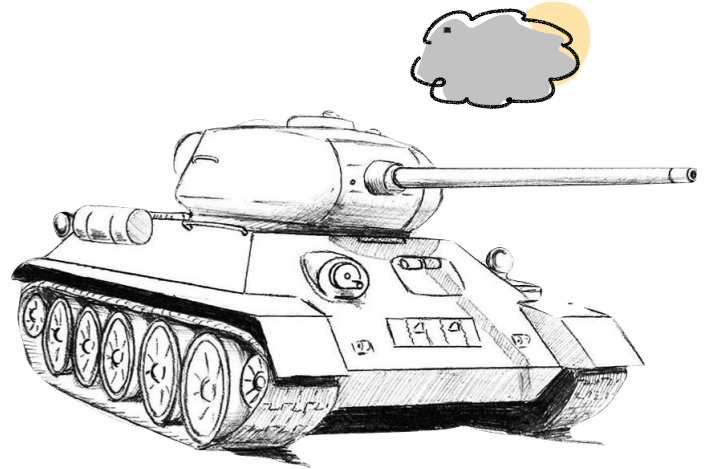
# MORE TECHNICALLY...

- We are going to use the **probabilistic model** of learning
- There is some probability distribution over example/label pairs called the **data generating distribution**
- **Both** the training data and the test set are generated based on this distribution

# TRAINING SET DESIGN

- The failure of a machine learning algorithm is often caused by a **bad selection of training samples**
- The issue is that we might introduce unwanted correlations from which the algorithm derives wrong conclusions

“The US Army trained a program to differentiate American tanks from Russian tanks with 100% accuracy. Only later did analysts realized that the American tanks had been photographed on a sunny day and the Russian tanks had been photographed on a cloudy day. The computer had learned to detect brightness.” [probably a legend]



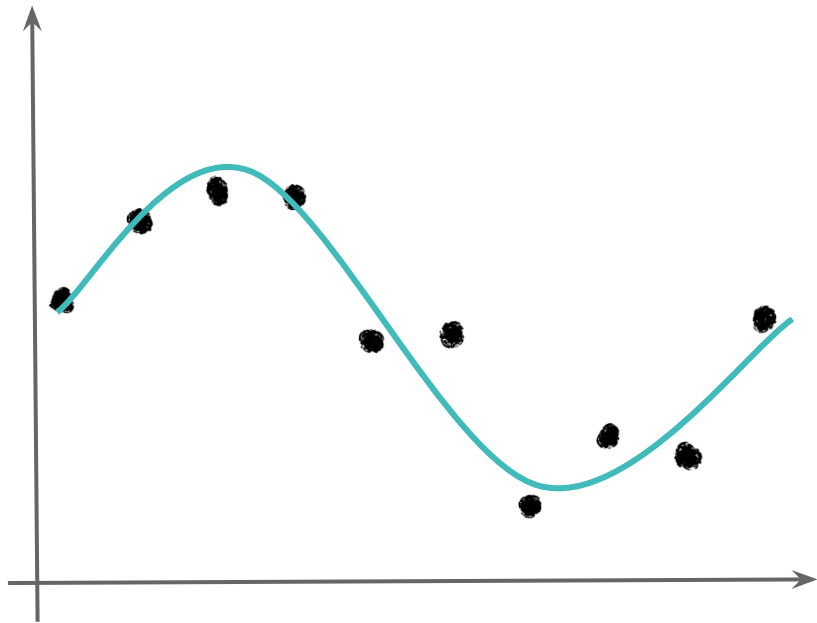
# EXAMPLE: POLYNOMIAL CURVE FITTING

## Data

$$T = \{(x_1, y_1), \dots, (x_N, y_N)\}$$

Data generated from  $\sin(2\pi x) + \text{noise}$

Training set with  $N = 10$  points

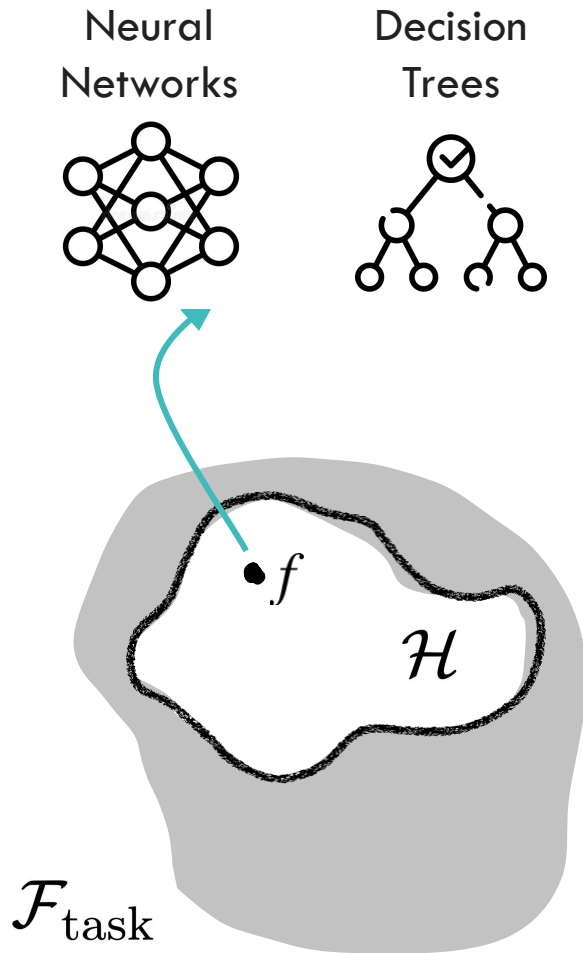


# MODEL AND HYPOTHESIS SPACE

- A model is like a “program” to solve the problem
- It is the implementation of a function  $f \in \mathcal{F}_{\text{task}}$  that can be tractably computed
- A set of models forms an hypothesis space

$$\mathcal{H} \in \mathcal{F}_{\text{task}}$$

- The learning algorithm seeks a solution within the hypothesis space



# EXAMPLE: POLYNOMIAL CURVE FITTING

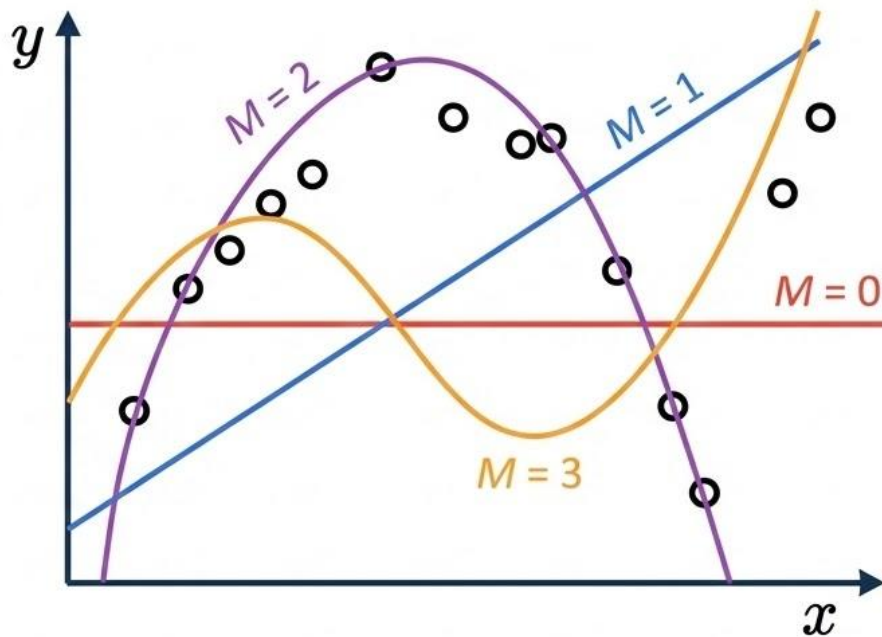
## Model

$$f_w(x) = \sum_{j=0}^M w_j x^j$$

Parameters

$$\mathcal{H}_M = \{f_w : w \in \mathbb{R}^M\}$$

Hypothesis space for fixed  $M$

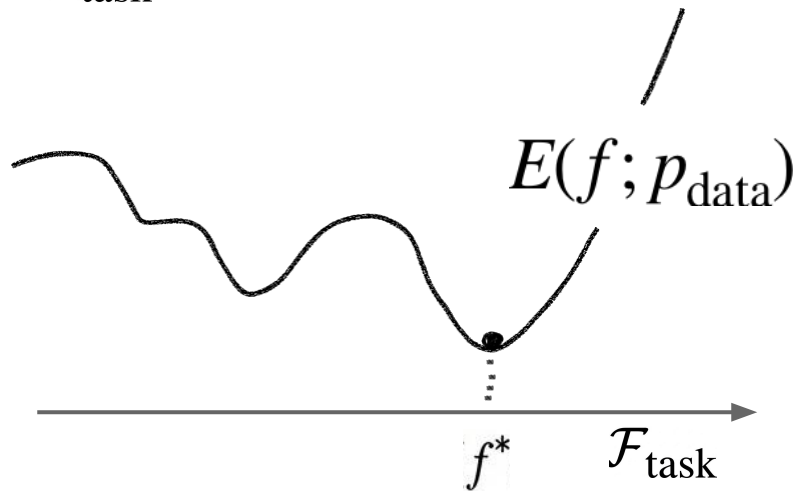


# OBJECTIVE - THE IDEAL TARGET

- Minimize a **(generalization) error function**  $E(f; p_{\text{data}})$
- It determines how well a solution  $f \in \mathcal{F}_{\text{task}}$  fits some given data
- Guides the selection of the best solution in  $\mathcal{F}_{\text{task}}$

$$f^* \in \arg \min_{f \in \mathcal{F}_{\text{task}}} E(f; p_{\text{data}})$$

Too large search space and we need an implementation.

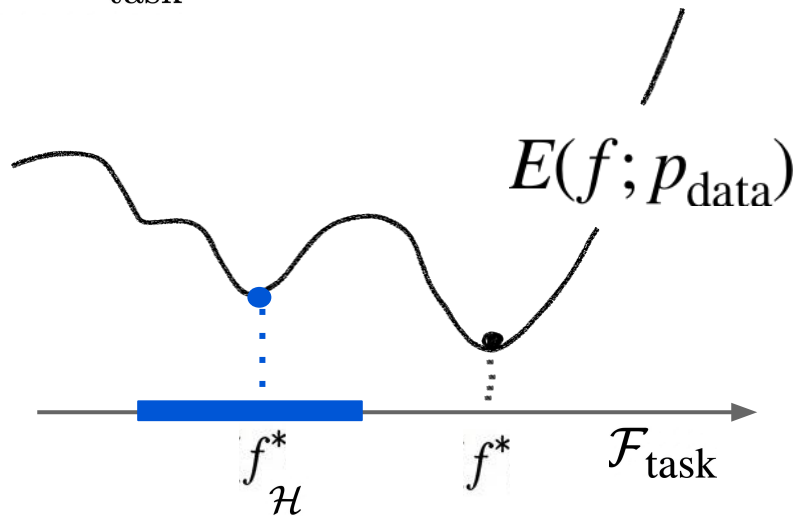


# OBJECTIVE - THE FEASIBLE TARGET

- We need to restrict the focus on finding functions that can be implemented and evaluated in a tractable way
- We define a model hypothesis space  $\mathcal{H} \in \mathcal{F}_{\text{task}}$  and seek a solution within that space

$$f_{\mathcal{H}}^* \in \arg \min_{f \in \mathcal{H}} E(f; p_{\text{data}})$$

Cannot be computed exactly,  
for  $p_{\text{data}}$  is unknown



# OBJECTIVE - THE ACTUAL TARGET

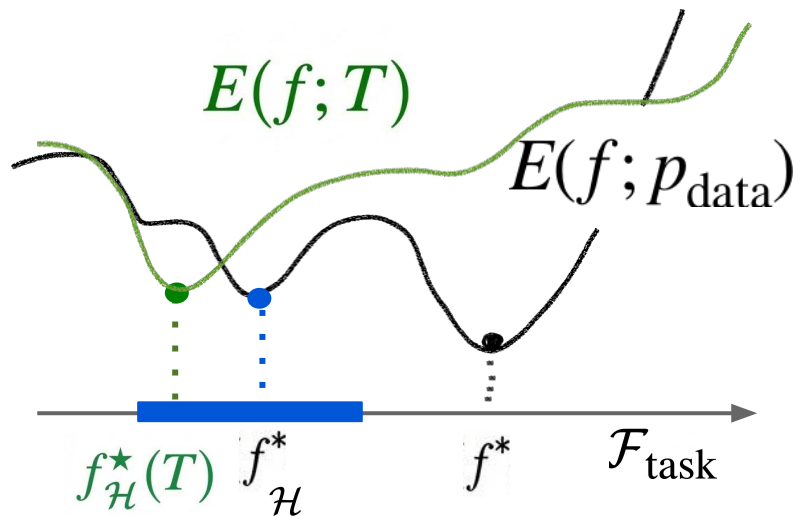
We need to work on a data sample, i.e. a **training set**  $T = \{(x_1, y_1), \dots, (x_N, y_N)\}$

$$(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$$

$$(x_i, y_i) \sim p_{\text{data}}$$

$$f_{\mathcal{H}}^{\star}(T) \in \arg \min_{f \in \mathcal{H}} E(f; T)$$

Training error



# ERROR FUNCTION

Typically the generalization and training error functions can be written in terms of a **pointwise loss**  $L(f; (x_i, y_i))$  measuring the error incurred by  $f$  on each training example  $(x_i, y_i)$

$$E(f; p_{\text{data}}) = \mathbb{E}_{(x,y) \sim p_{\text{data}}} [L(f; (x, y))]$$



Generalization error

# ERROR FUNCTION

Typically the generalization and training error functions can be written in terms of a **pointwise loss**  $L(f; (x_i, y_i))$  measuring the error incurred by  $f$  on each training example

$$E(f; T) = \frac{1}{N} \sum_{i=1}^N L(f; (x_i, y_i))$$



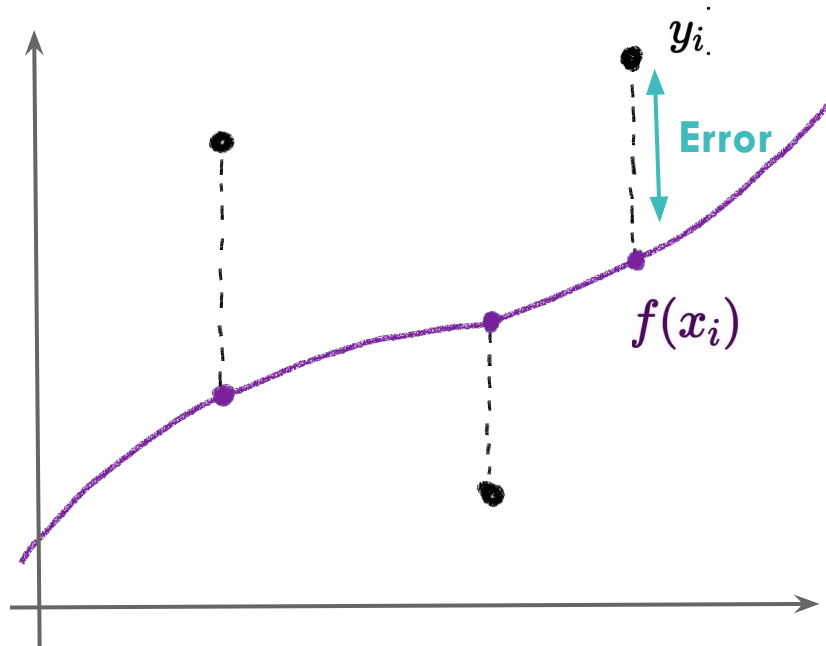
Training error

# EXAMPLE: POLYNOMIAL CURVE FITTING

## Error function

$$E(f; T) = \frac{1}{N} \sum_{i=1}^N [f(x_i) - y_i]^2$$

Pointwise loss



# EXAMPLE: POLYNOMIAL CURVE FITTING

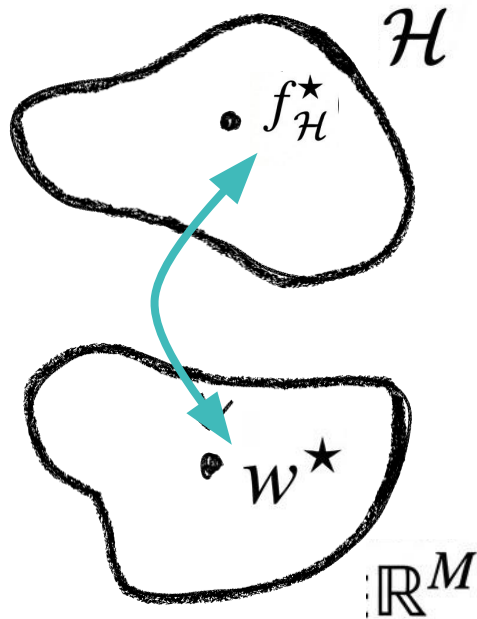
Objective

$$f_{\mathcal{H}}^{\star}(T) \in \arg \min_{f \in \mathcal{H}} E(f; T)$$

equivalent to:

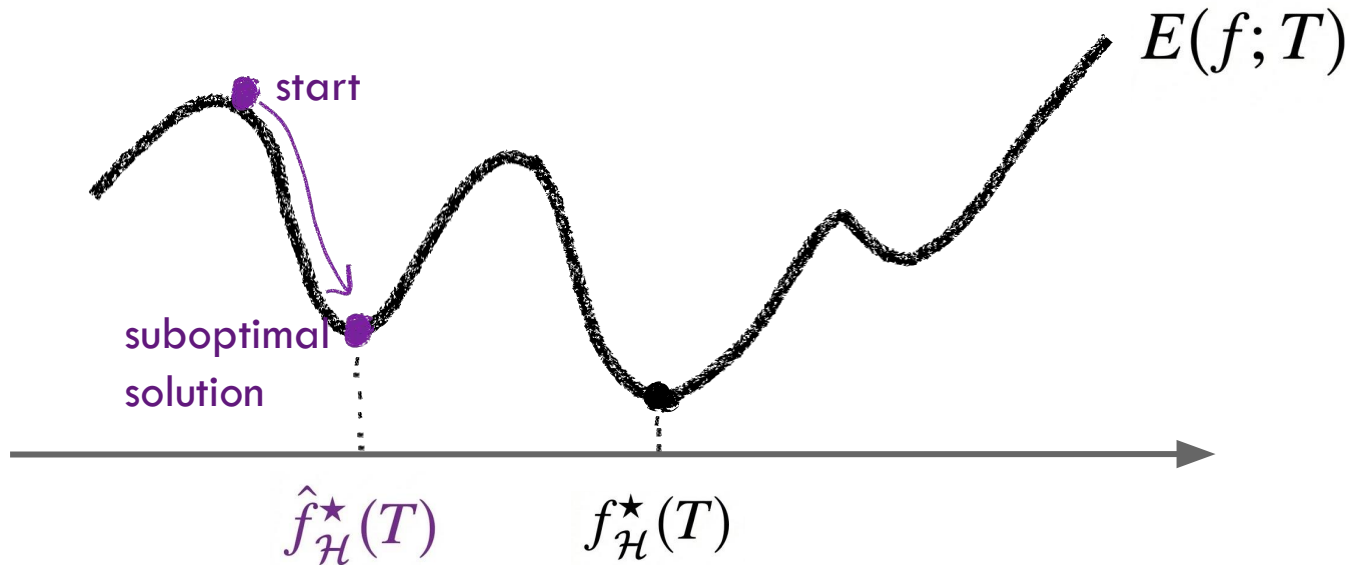
$$w^{\star} \in \arg \min_{w \in \mathbb{R}^M} \frac{1}{N} \sum_{i=1}^N [f_w(x_i) - y_i]^2$$

Requires solving a linear system of equations



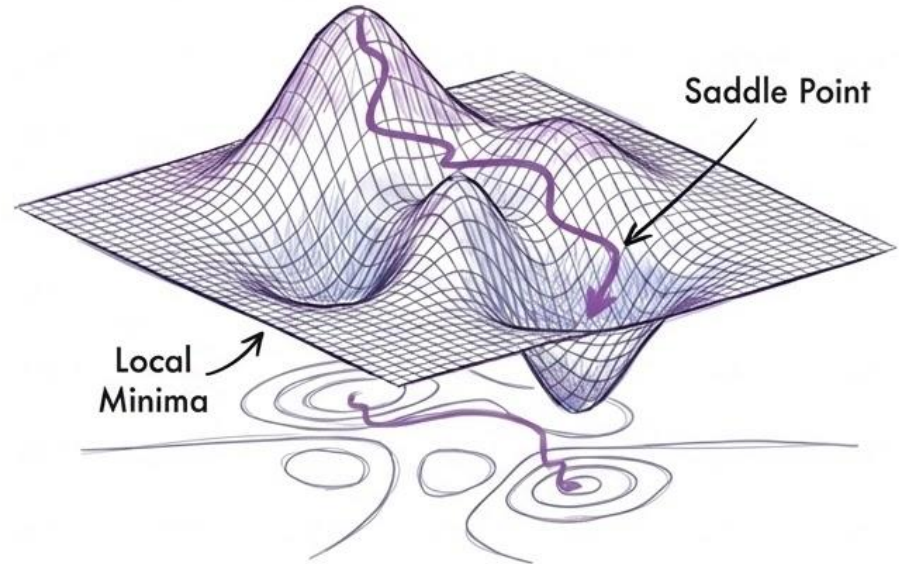
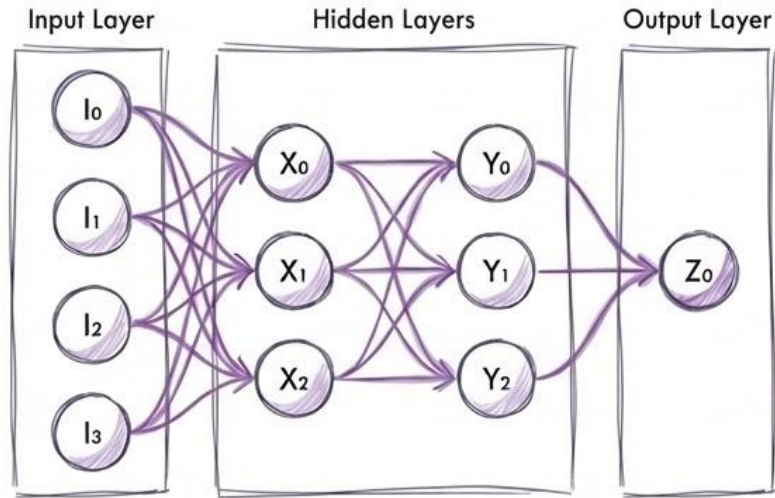
# LEARNING ALGORITHM

Solves the optimization problem targeting  $f_{\mathcal{H}}^{\star}(T)$  but might end up in a different result



# EXAMPLE: NEURAL NETWORK OPTIMIZATION

Issues with optimization: saddle points, local minima



RECAP

$\mathcal{H}$

$E(f; T)$

$E(f; p_{\text{data}})$

**Generalization  
error**

**Training error**

$f_{\mathcal{H}}^{\star}$

**Actual  
target**

$f_{\mathcal{H}}^{\star}(T)$

**Feasible  
target**

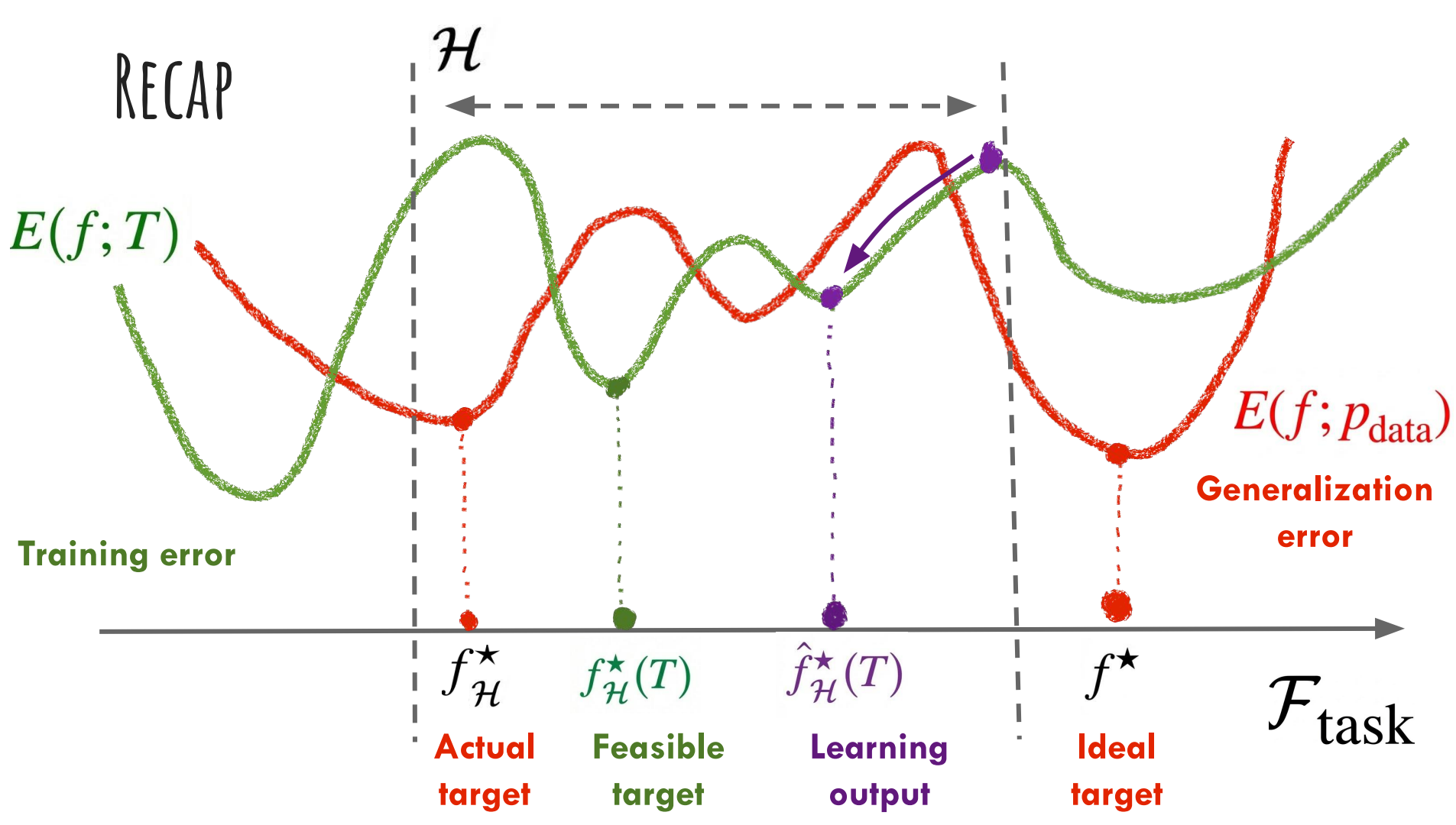
$\hat{f}_{\mathcal{H}}^{\star}(T)$

**Learning  
output**

$f^{\star}$

**Ideal  
target**

$\mathcal{F}_{\text{task}}$

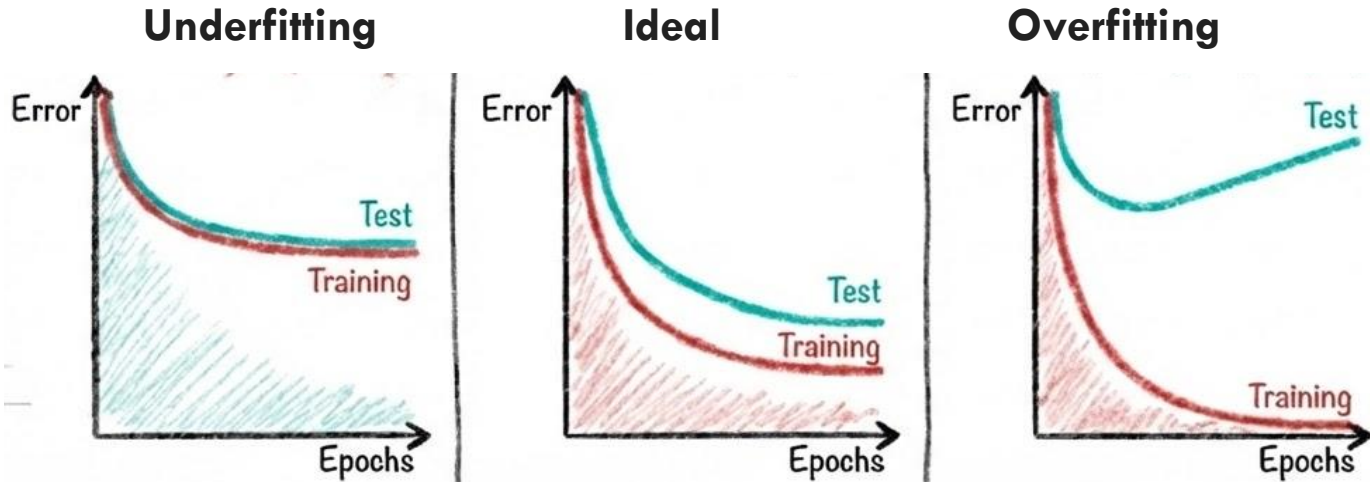


# OVERFITTING & UNDERFITTING

- **Training Error**: Error measured on the training dataset.
- **Generalization Error** (proxy: test error): Error measured on unseen data.
- Goal: Minimize generalization error, not just training error.

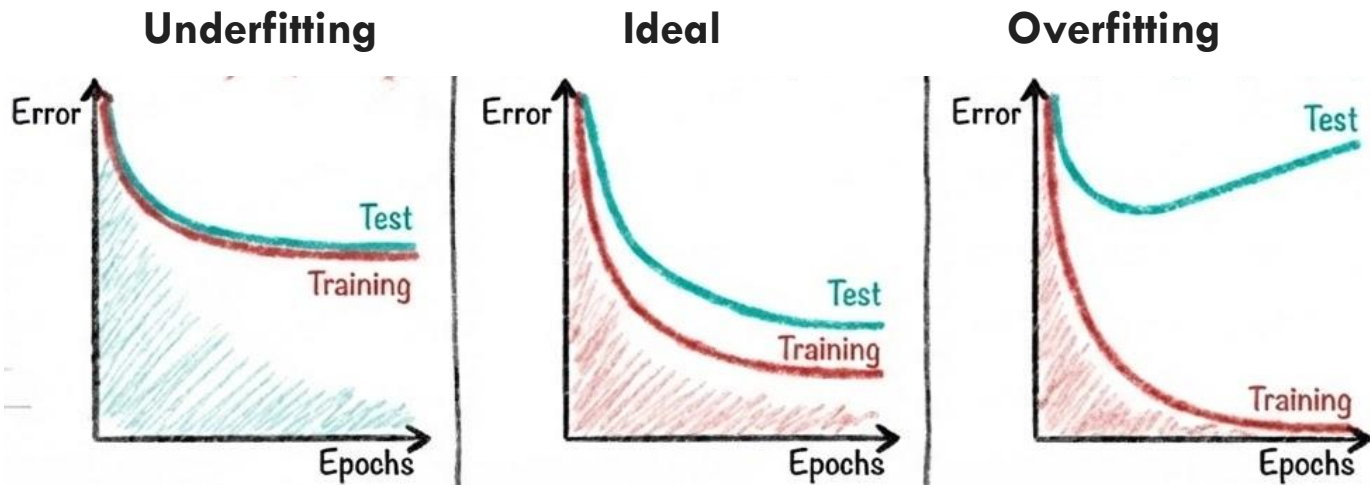
# UNDERFITTING

- Model is too simple to capture underlying patterns.
- High training error
- High generalization error
- Poor performance on both training and test data.



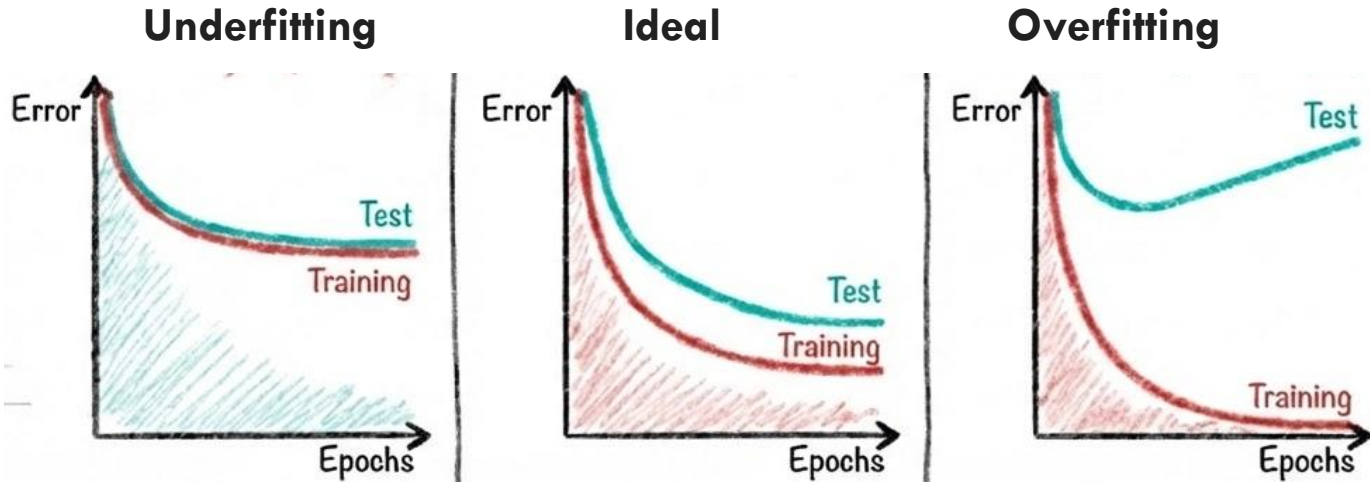
# OVERFITTING

- Model is too complex and memorizes training data.
- Very low training error
- High generalization error
- Performs well on training data but poorly on unseen data.



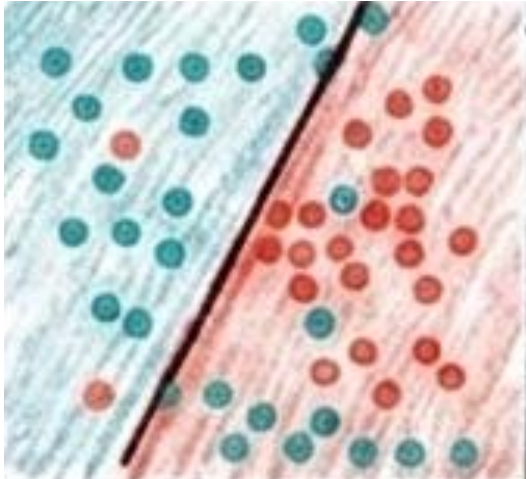
# IDEAL

- Low training error
- Low generalization error
- Small gap between training and test error

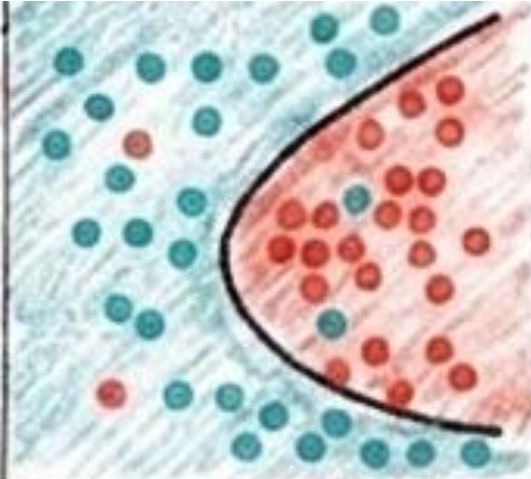


# OVERFITTING & UNDERFITTING: CLASSIFICATION

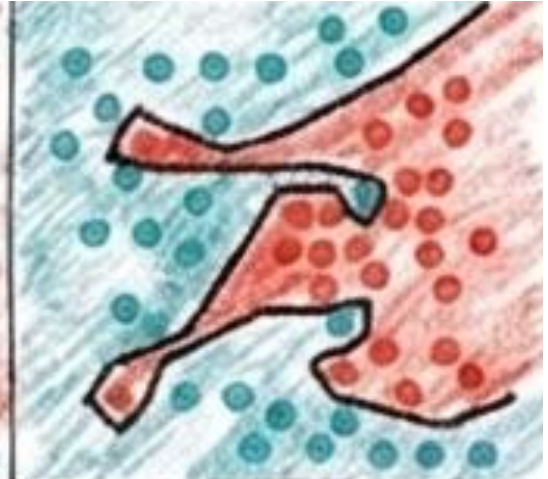
**Underfitting**



**Ideal**

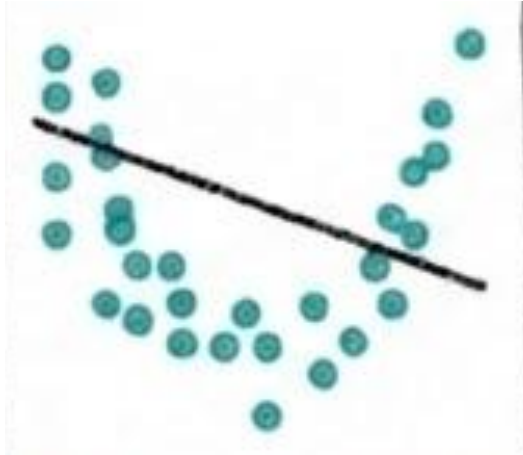


**Overfitting**



# OVERFITTING & UNDERFITTING: REGRESSION

**Underfitting**



**Ideal**



**Overfitting**



APPROXIMATION ERROR

$\mathcal{H}$

Error induced by  
the hypothesis  
space

$E(f; p_{\text{data}})$

Generalization  
error

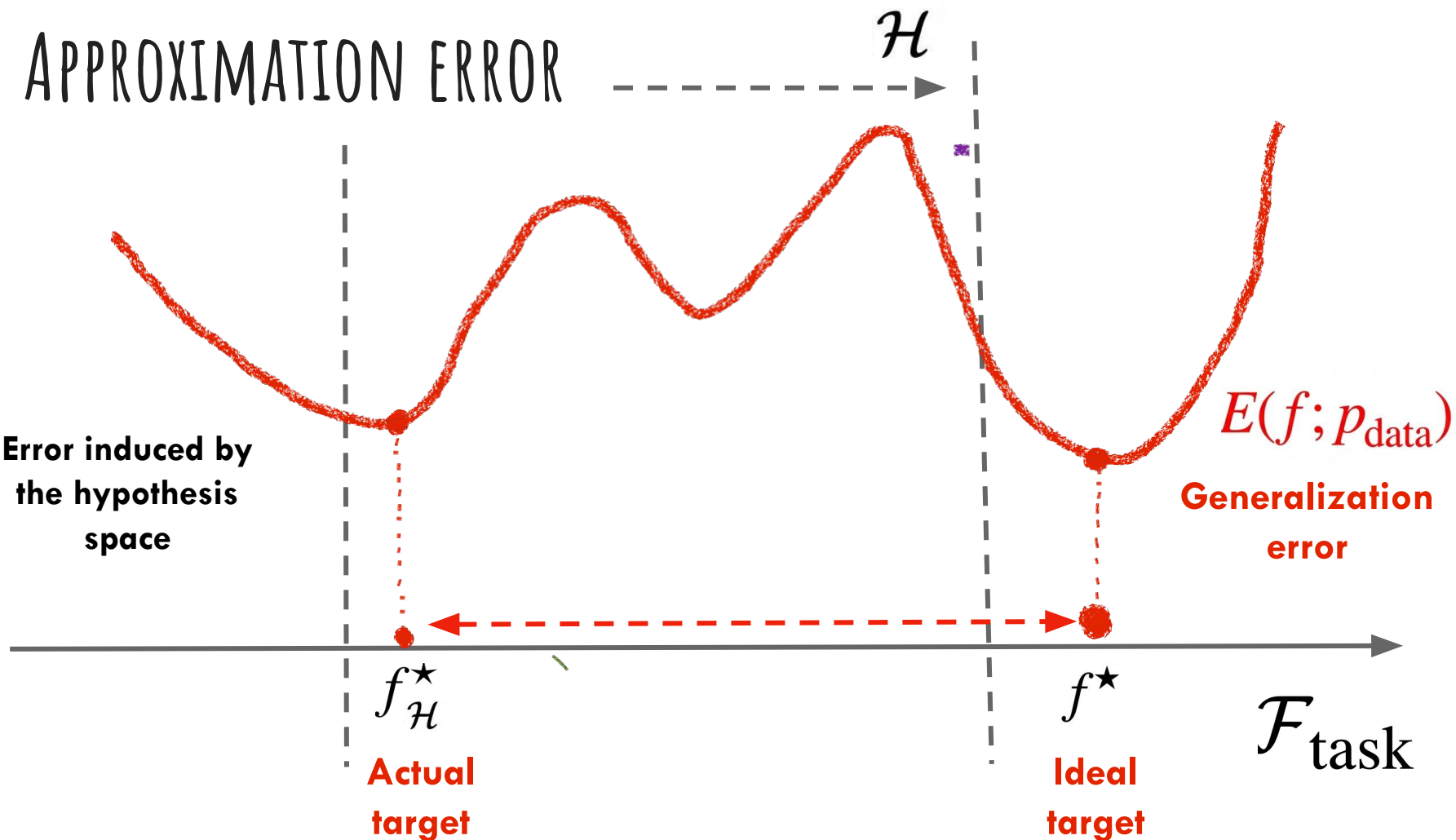
$f_{\mathcal{H}}^*$

Actual  
target

$f^*$

Ideal  
target

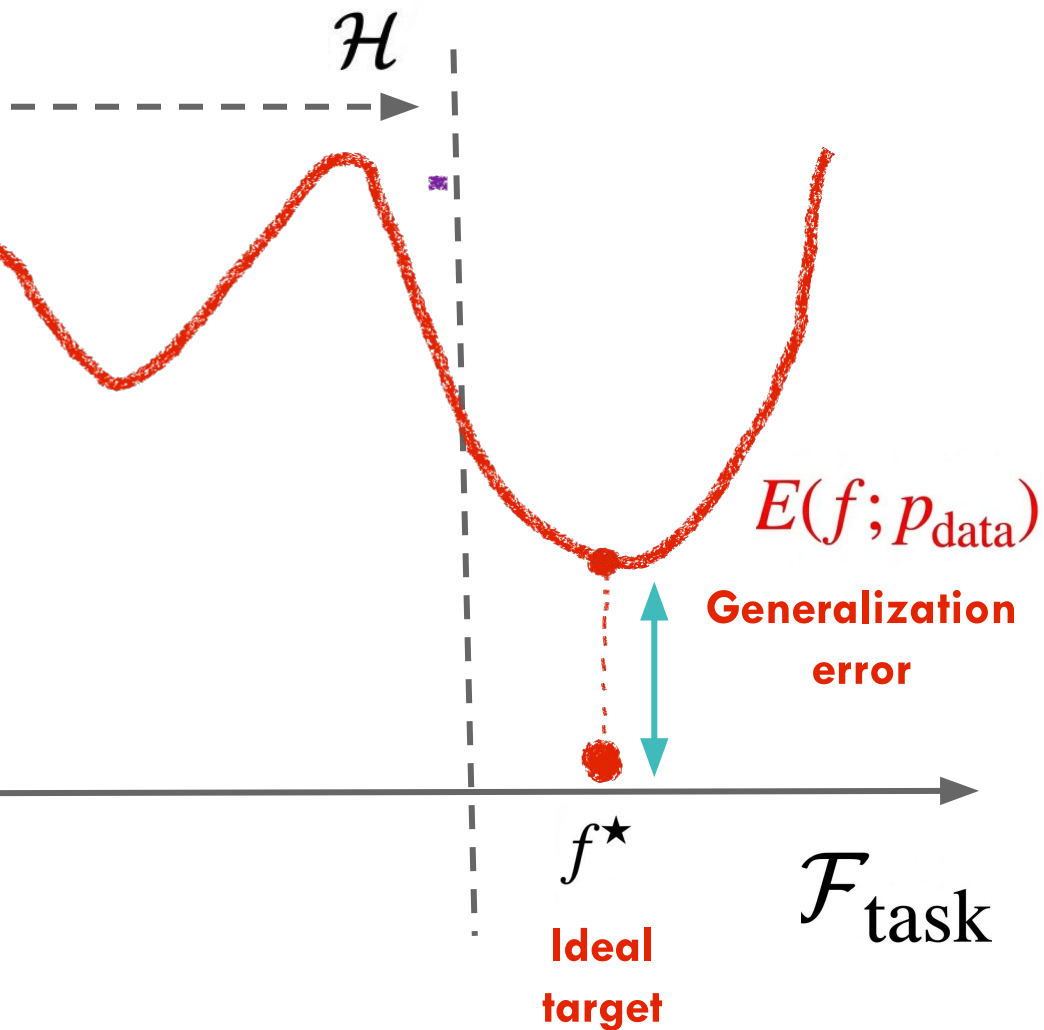
$\mathcal{F}_{\text{task}}$



# IRREDUCIBLE ERROR

Error due to  
inherent  
variability

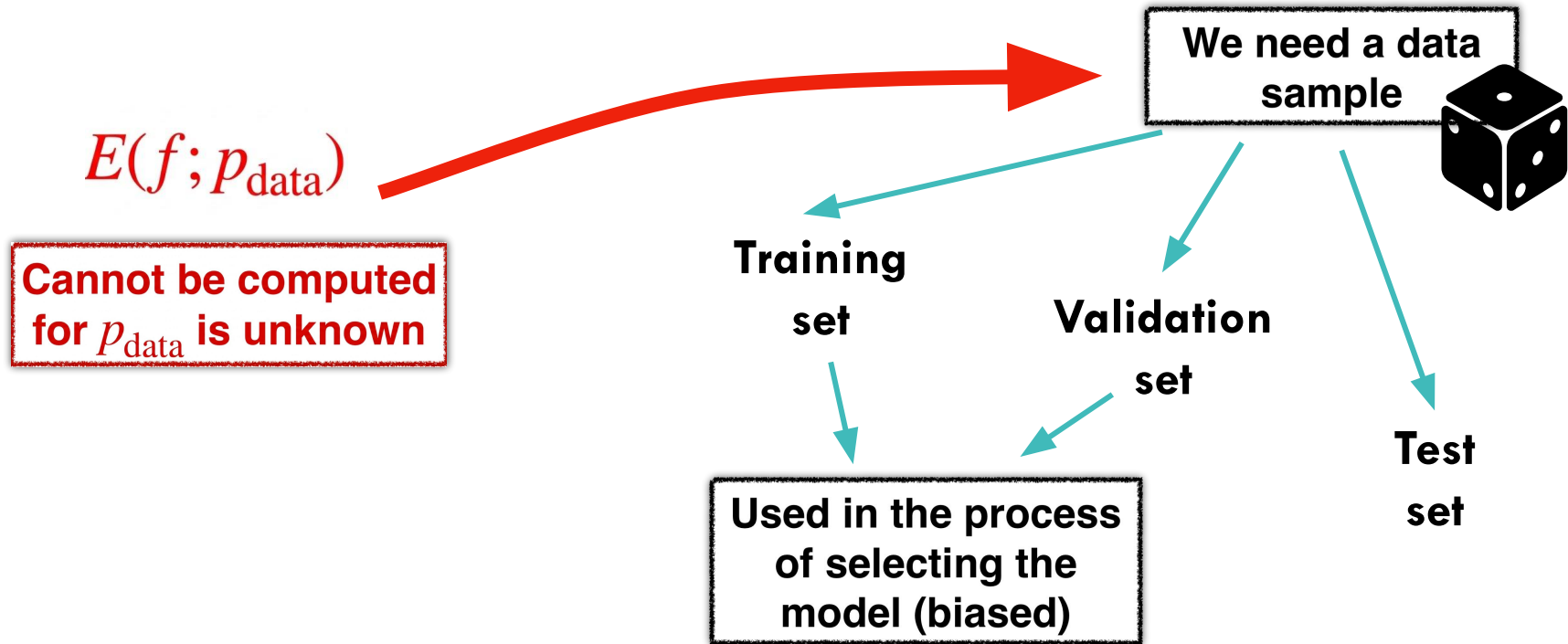
**Irreducible errors:** it is an error that we cannot reduce by choosing a better model. it is due to randomness or natural variability in a system. These are deterministic factors that you don't observe, but which could be interpreted as inherent variability.



# IRREDUCIBLE ERROR

- It is the portion of prediction error that cannot be eliminated, no matter how powerful the model is or how much data you collect.
- Even the best model cannot predict:
  - Measurement errors
  - Hidden/unobserved variables
  - Intrinsic randomness (e.g., human behavior)
- So irreducible error represents the **fundamental uncertainty** in the problem.

# HOW TO ESTIMATE GENERALIZATION ERROR

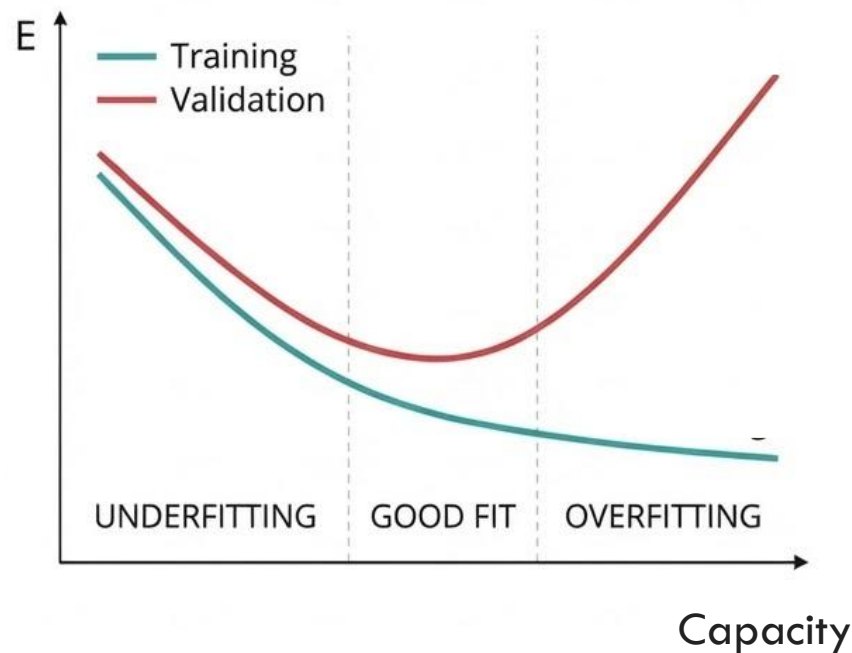


# MODEL CAPACITY

- **Model capacity:** ability of a model to fit a wide variety of functions.
  - High capacity: can represent very complex patterns.
  - Low capacity: limited to simple patterns.
- Controlled by:
  - Number of parameters
  - Model architecture (e.g., depth of neural network)
  - Feature dimensionality

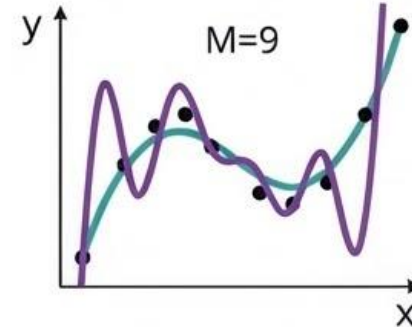
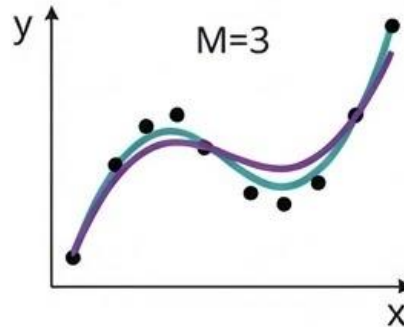
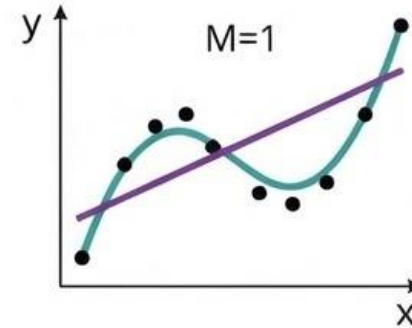
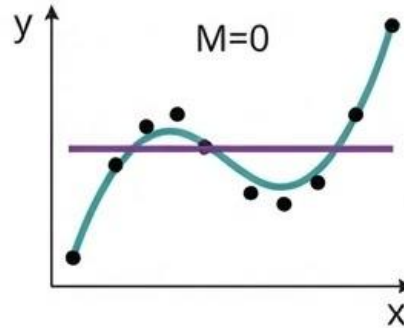
# MODEL CAPACITY

- **Low capacity** is associated to underfitting
  - High training error
  - High generalization error
- **High capacity:** Risk of Overfitting
  - Very low training error
  - High generalization error
- Model memorizes noise instead of learning structure



# EXAMPLE: POLYNOMIAL CURVE FITTING

Solutions for  
different model  
complexities



ideal  
estimated

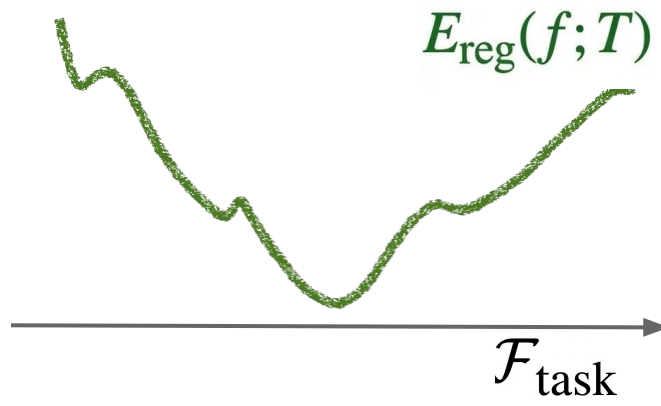
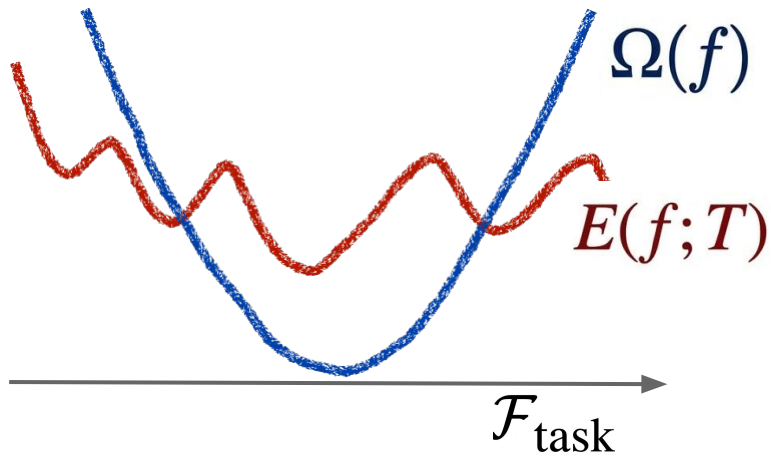
# HOW TO IMPROVE GENERALIZATION

- Avoid attaining the minimum on training error
- Reduce model capacity
- Change the objective with a regularization term
- Inject noise in the learning algorithm
- Add more training samples, e.g. augment the training set with transformations
- Combine predictions from multiple, decorrelated models (ensembling)

# REGULARIZATION

Modification of the training error function with a term  $\Omega(f)$  that typically penalizes complex solutions

$$E_{\text{reg}}(f; T) = E(f; T) + \lambda \Omega(f)$$

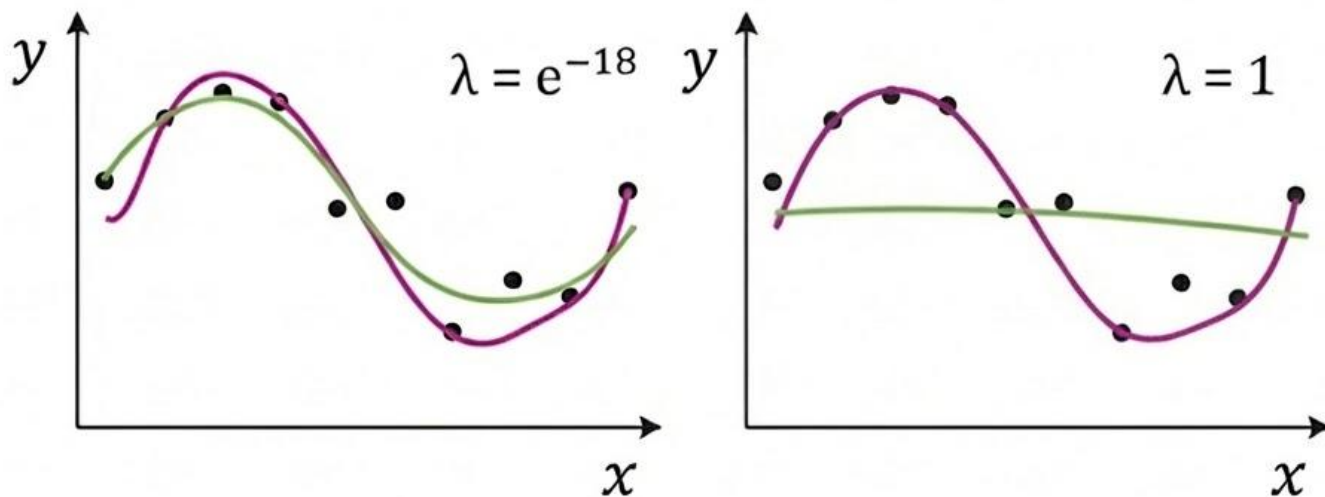


# EXAMPLE: POLYNOMIAL CURVE FITTING

We regularize by penalizing polynomials with large coefficients

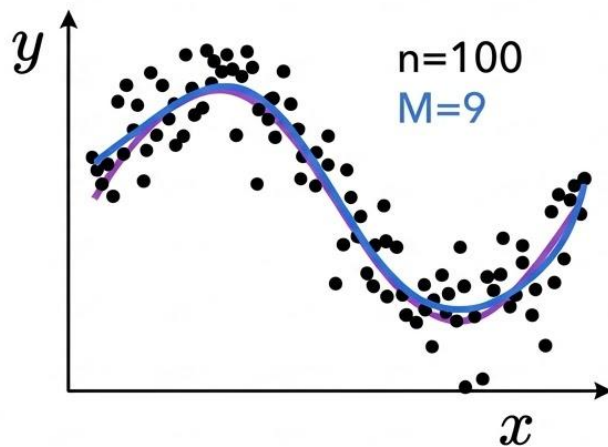
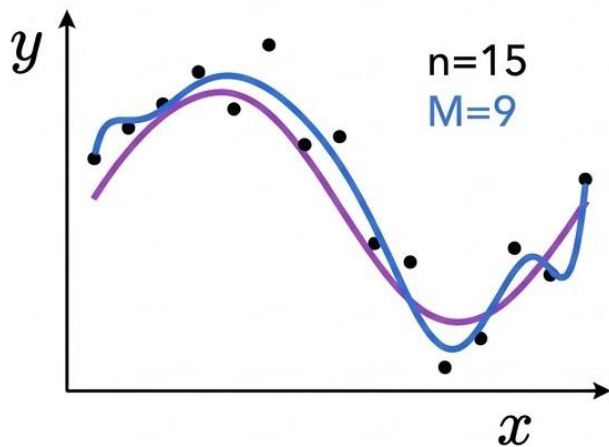
$$E_{\text{reg}}(f_w; T) = \frac{1}{N} \sum_{i=1}^N [f_w(x_i) - y_i]^2 + \frac{\lambda}{N} \|w\|^2$$

# EXAMPLE: POLYNOMIAL CURVE FITTING



# AUGMENTING TRAINING SET: POLYNOMIAL CURVE FITTING

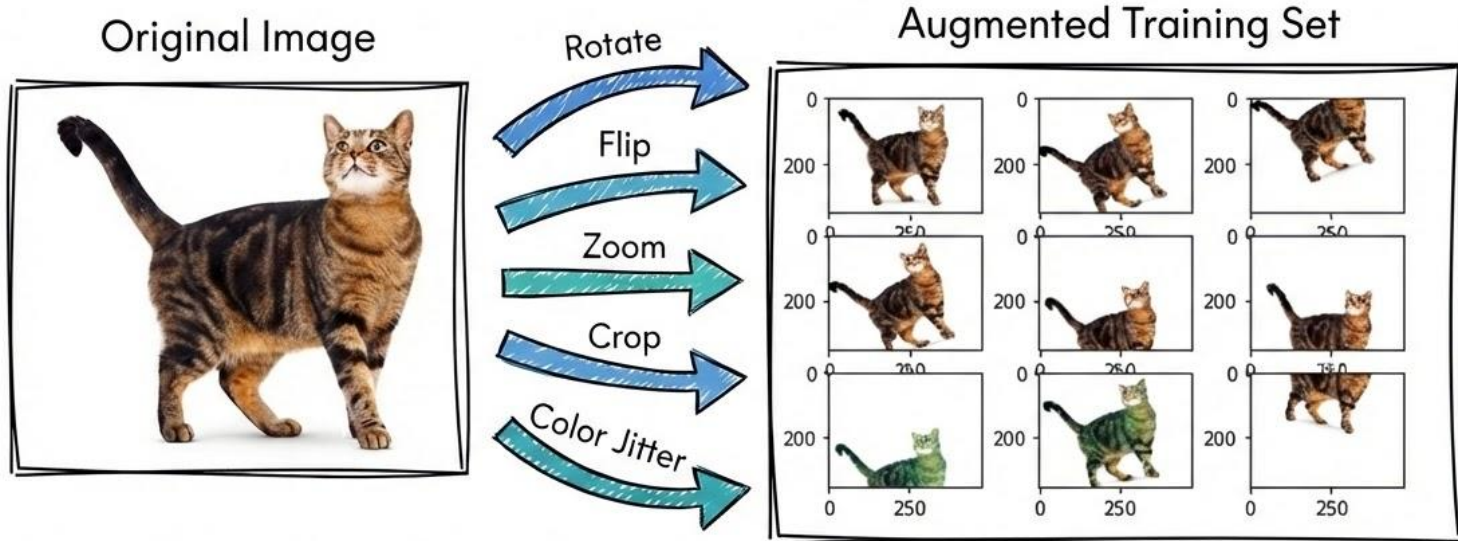
## Generalization vs data size



$$E(f; T) \rightarrow E(f; p_{\text{data}}) \text{ as } n \rightarrow \infty$$

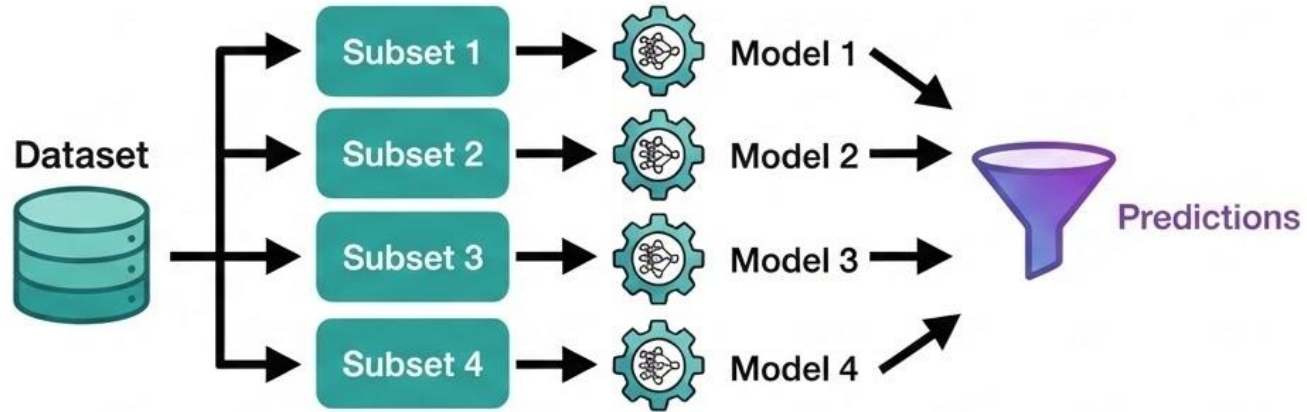
# AUGMENTING THE TRAINING SET

Use transformations (e.g. data augmentation in neural networks)

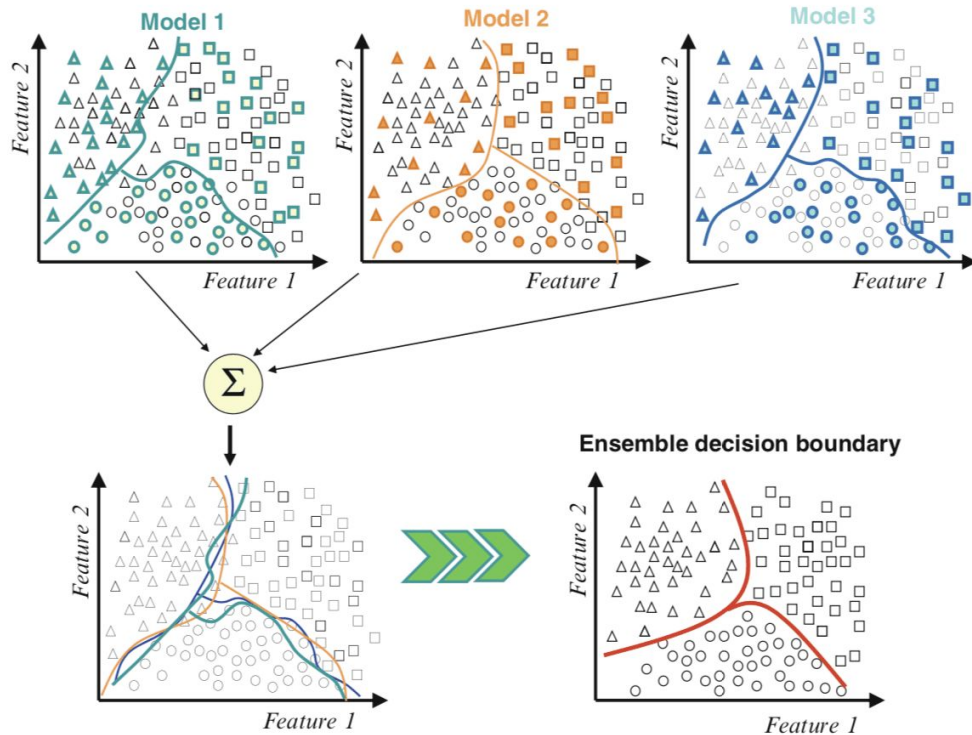


# COMBINE PREDICTIONS FROM MULTIPLE MODELS

Should be decorrelated models



# COMBINE PREDICTIONS FROM MULTIPLE MODELS



# QUESTIONS?

